

**LAPORAN PRAKTIKUM
PEMROGRAMAN PERANGKAT BERGERAK**

PERTEMUAN 13 IMPLEMENTASI PROVIDER NETWORKING

API PERANGKAT KERAS



Disusun Oleh :

Alden Audy Akbar

Dosen Pengampu :

Cahyo Prihantoro, S.Kom., M.Eng.

**LABORATORIUM HIGH PERFORMANCE FAKULTAS INFORMATIKA
UNIVERSITAS TELKOM PURWOKERTO**

2026

Source code

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

void main() {
  runApp(
    // Mendaftarkan ChangeNotifierProvider di tingkat akar aplikasi
    ChangeNotifierProvider(
      create: (context) => CounterProvider(),
      child: const MyApp(),
    ),
  );
}

// =====
// STATE MANAGEMENT: COUNTER PROVIDER
// =====

class CounterProvider with ChangeNotifier {
  int _counter = 0;

  int get counter => _counter;

  void incrementCounter() {
    _counter++;
    notifyListeners(); // Memberitahu semua widget yang mendengarkan untuk
    memperbarui UI
  }
}

// =====
// APLIKASI UTAMA
// =====

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Tugas Modul 12-13 Provider & Notifikasi',
      theme: ThemeData(
```

```

        colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepPurple),
        useMaterial3: true,
      ),
      home: const HalamanUtamaCounter(),
    );
  }
}

```

```

class HalamanUtamaCounter extends StatelessWidget {
  const HalamanUtamaCounter({super.key});

```

```

  // Fungsi internal untuk memicu sistem notifikasi lokal melayang
  void _picuNotifikasiLokal(BuildContext context, int nilaiTerbaru) {
    ScaffoldMessenger.of(context).removeCurrentSnackBar();
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(
        content: Row(
          children: [
            const Icon(Icons.notifications_active, color:
Colors.amberAccent),
            const SizedBox(width: 12),
            Expanded(
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                mainAxisAlignment: MainAxisAlignment.min,
                children: [
                  const Text(
                    'Counter Update',
                    style: TextStyle(fontWeight: FontWeight.bold, fontSize:
14),
                  ),
                  Text(
                    'Nilai counter saat ini: $nilaiTerbaru',
                    style: const TextStyle(fontSize: 12),
                  ),
                ],
              ),
            ),
          ],
        ),
      ),
    );
  }
}

```

```
    ),  
    duration: const Duration(seconds: 2),  
    backgroundColor: Colors.deepPurple.shade800,  
    behavior: SnackBarBehavior.floating, // Membuat notifikasi melayang  
    di atas UI
```

```
        shape: RoundedRectangleBorder(borderRadius:  
BorderRadius.circular(10)),  
    ),  
);  
}
```

```
@override  
Widget build(BuildContext context) {  
    return Scaffold(  
        appBar: AppBar(  
            title: const Text('Provider & Notifikasi'),  
            backgroundColor: Theme.of(context).colorScheme.inversePrimary,  
            centerTitle: true,  
        ),  
        body: Center(  
            child: Column(  
                mainAxisAlignment: MainAxisAlignment.center,  
                children: [  
                    const Text(  
                        'Anda telah menekan tombol sebanyak:',  
                        style: TextStyle(fontSize: 16),  
                    ),  
                    const SizedBox(height: 10),  
  
                    // Menggunakan Consumer untuk mendengarkan perubahan nilai dari  
                    Provider  
                    Consumer<CounterProvider>(  
                        builder: (context, counterProvider, child) {  
                            return Text(  
                                '${counterProvider.counter}',  
                                style: const TextStyle(fontSize: 48, fontWeight:  
FontWeight.bold),  
                            );  
                        },  
                    ),  
                ],  
            ),  
        ),  
    );  
}
```

```

    ],
  ),
),
floatingActionButton: FloatingActionButton(
  onPressed: () {
    // 1. Memanggil fungsi increment dari Provider tanpa mendengarkan
ulang seluruh build
    final provider = Provider.of<CounterProvider>(context, listen:
false);

    provider.incrementCounter();

    // 2. Memicu notifikasi dengan mengirimkan nilai counter terbaru
    _picuNotifikasiLokal(context, provider.counter);
  },
  tooltip: 'Tambah',
  child: const Icon(Icons.add),
),
);
}
}

```

Hasil:



1. Mekanisme *State Management* (Provider)

Penggunaan *Provider* bertujuan untuk memisahkan lapisan logika bisnis dari lapisan antarmuka pengguna (*UI*), sehingga pengelolaan data menjadi lebih terstruktur:

- **ChangeNotifier (CounterProvider):** Berperan sebagai pusat penyimpanan *state* (data) angka. Di dalamnya terdapat fungsi-fungsi untuk memanipulasi variabel *counter*. Setiap perubahan nilai diakhiri dengan pemanggilan `notifyListeners()` untuk memperbarui data secara sistematis.
- **notifyListeners:** Berfungsi sebagai sistem transmisi sinyal yang menginformasikan kepada seluruh bagian aplikasi bahwa telah terjadi perubahan data (*state update*).
- **ChangeNotifierProvider:** Ditempatkan pada *root* (akar) aplikasi agar *instance* pengelola data dapat diakses secara global oleh seluruh hierarki *widget* di bawahnya.

- Consumer: *Widget* reaktif yang memantau perubahan data pada *Provider*. Saat menerima sinyal pembaruan, Consumer akan memicu *rebuild* (pembangun ulang) hanya pada area *widget* yang terbungkus di dalamnya, sehingga proses *rendering* jauh lebih efisien dibandingkan menyegarkan seluruh halaman.

2. Sistem Notifikasi *Event-Driven*

Sistem notifikasi pada aplikasi ini dirancang secara *event-driven*, di mana pesan muncul sebagai respons langsung atas interaksi pengguna:

- Pemicu Aksi: Setiap kali tombol aksi ditekan, *Provider* akan memperbarui nilai data secara *real-time* dan menyiapkan nilai terbaru untuk ditampilkan.
- Injeksi Data: Nilai *integer* terkini dari *counter* diintegrasikan secara dinamis ke dalam parameter teks notifikasi, sehingga pengguna selalu mendapatkan informasi yang akurat.
- Manajemen Antrean: Sebelum menampilkan pesan baru, sistem akan memanggil `removeCurrentSnackBar()`. Metode ini krusial untuk memastikan tidak terjadi penumpukan notifikasi (*message queuing*) yang mengganggu kenyamanan pengguna.
- Visualisasi Melayang: Menggunakan `SnackBarBehavior.floating` yang memberikan efek notifikasi melayang di atas *UI*. Hal ini memastikan pesan tidak tertutup oleh komponen lain, serta dilengkapi dengan judul dan ikon sebagai indikator visual untuk memberikan umpan balik yang informatif.